

REST Resources

Prof. Dr. Peter Braun

19. November 2024

1 Titlepage

REST Resources

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- You learn about resources of Web APIs
- You learn how to design good URIs
- You can apply different resource representations

Welcome to this unit about resources in Web APIs. In the previous units, you have gained an initial understanding of how a web-based API should be structured compared to a more function-oriented API. We talk about a web-based API when the communication is done over the HTTP protocol, the four HTTP verbs are used in terms of the four basic CRUD operations, and we exchange documents between client and server. In this unit, we will discuss what we mean by resources, how we address resources via a URI, and what representations of resources are. In Web APIs, resources are typically provided in different representations, such as JSON and XML. We will discuss how to provide XML in a service implemented with JAX-RS parallel to JSON.

3 A Word of Warning

A Word of Warning

- Web APIs must be seen on top of the HTTP protocol
- Web APIs and REST APIs follow design principles
- Frameworks only support the HTTP protocol
- Developers have a lot of freedom in designing APIs

Let's take a moment to highlight an important consideration when working with Web APIs. First, it's crucial to understand that Web APIs operate on top of the HTTP protocol. This means the design and functionality of your API are inherently tied to how HTTP handles requests, responses, and status codes. However, both Web APIs and REST APIs follow certain design principles, such as stateless communication, proper use of HTTP methods, and meaningful status codes. These principles ensure that APIs are consistent, predictable, and easy to consume. While frameworks like JAX-RS or Spring provide powerful tools to implement these principles, they only offer support for the HTTP protocol itself. The API design—including how resources are structured, endpoints are named, and methods are chosen—is entirely up to the developer. This brings us to a critical point: developers have a lot of freedom in designing APIs. More freedom than when designing RPC-like APIs. While this is empowering, it is also responsible for following best practices and maintaining consistency. Poorly designed APIs can lead to clarity, efficiency, and increased maintenance costs.

4 Resources

Resources

- Everything what can be named is a resource
- Models of real things like users, books, rooms
- Models of virtual things like messages, images, courses, bank accounts
- Resources can be named as nouns – not as verbs
- Every resource has a unique identifier
- Every resource can be accessed using a URI

Let's start by discussing in a little more detail what these documents are. In the first place, we exchange them between the client and the server in a web-based distributed system. Instead of the word "document" or "data," from now on, we will use the term "resources." Suppose you want to determine the necessary resources for your application. In that case, you can use techniques similar to those you may already be familiar with from object-oriented analysis or database modeling. Resources in the sense of Web APIs – and classes or objects in the sense of object-oriented analysis – or entities in the sense of analysis for databases – have a lot of similarities. Any real-world thing that you can give a name to is, first of all, a resource. If your distributed system is about managing books, there is most likely a resource called "book." If it is about managing rooms, there is a resource named "room." Of course, there may be virtual things that you need to model as resources in your system—for example, messages, images, and modules in the context of a university or bank account. Every resource has a name, which must be a noun, not a verb. Each resource also needs an artificial or natural way to create identifiers because you need to address each resource individually. And finally, for each resource, you need a URL to reach your resource on the server.

5 Unique Identifiers

Unique Identifiers

- Mostly created by the database management system
 - `CREATE TABLE user(id int, ..., PRIMARY KEY(id))`
 - In NoSQL databases keys hash values like 5162fba3a...
-
- Which one is better?
 - How likely is it to guess a correct identifier?

A few words about the problem of what a unique identifier might be. If you've ever worked with a database management system, you already know this concept. Every resource ends up being a record in a database management system, and as you know, each record needs a unique ID so that you can address the record directly. For example, this unique ID can be a sequential number assigned by the database itself. However, such a sequential number is relatively easy to guess, and in some applications, you would like to avoid that clients can guess such IDs. Therefore, you can often find an approach with random IDs, such as hash values. There are also cases where a unique ID results naturally. For example, the email address can be used as a unique identifier; in the case of books, for example, there is a unique serial number. Note, however, that this serial number does not uniquely identify each book copy but rather the set of all books of this kind. For example, in a library, if you need to distinguish each book copy, you need yet another type of unique identifier.

6 Design URIs for Resources

Design URIs for Resources

- `http://example.org/users/11`
- `http://example.org/users/james@bond.com`
- It is undefined if singular or plural should be used
- Personally, I prefer plural such as `users`

Now that we have defined the names for the resources and know what the unique identifiers look like, we can next think about the URIs – i.e., the addresses clients use to work with resources on the server. For example, if we have a resource called `user`, then I suggest the path element under which the individual resources of type `User` are accessible is then called `“users.”` I prefer the plural form here because it is natural for us to request the collection of all users under the URI `users`. It would be strange to request this collection at `user`. If we know the ID of a specific user, then we append that unique identifier to the URI and can then access that unique record via `“users/11.”` If we used the email address as the unique identifier, we would write `“Users/james@bond.com”` in the URI.

7 Design URIs for Primary Resources: the wrong way

Design URIs for Primary Resources: the wrong way

- `http://example.org/getauthor/11`
- `http://example.org/author?id=11`
- `http://example.org/accounts/11?action=get`
- `http://example.org/author/11.json`

We can already make numerous mistakes when defining the addresses for our resources. I would like to briefly show some counterexamples, which are also used in this way in practice. Let's look at the counterexamples one after the other. In the first case, the path element, which should normally contain only the resource's name, also includes the HTTP verb. This is unnecessary because the HTTP verb is in a separate place in the HTTP request message. In the second example, a resource is not addressed via its unique ID, but a filter request is made, which can be recognized by the ?. In the meaning of HTTP, when building such URIs, you want to query all resources of the type author. Still, you want to apply a filter via the attribute ID, and specify the value of the desired element as a parameter, in this case, 11. Since the ID is unique, no filter query must happen here, but you can directly address the resources uniquely by adding the ID as the last path element. A filter query would be correct if, for example, you want to query all authors with a certain first name. Again, in the third example, a filter criterion is used. In this case, the parameter is Action, and the value is GET. This URI means resource 11 of type Account should be retrieved using a GET request. This is as wrong as in the first example because the HTTP verb GET is already in the HTTP request message. The fourth counterexample contains as a suffix the name of the representation format, in this case, JSON, in which the client would like to have the resources of type author with ID 4711. It is easy to think where this notation comes from because we all use such a suffix in file names to make the file format even more recognizable. Dot DOC, dot PNG, dot PDF, etc. Nevertheless, this suffix is wrong at this point because the data format in which the client wants the resources displayed is specified exactly in the HTTP request message via the request parameter "Accept." All these examples show solutions software developers devised without knowing the basics of the HTTP protocol and its

possibilities for configuring HTTP request messages. If they had studied the standards and specifications of HTTP a bit more, they could have found the correct solution.

8 Collection Resources

Collection Resources

- A collection of resources is also a resource
- `http://example.org/users`
- Used to GET all resources
- Used to POST (create) a new resource

So far, we have dealt with addressing individual resources and always used the unique identifier of the resources for this purpose. However, clients also want to be able to query collections of resources. Such a collection of individual resources is again considered a resource in a Web-based system, for which there is a separate type of representation. For example, in JSON, one uses the square brackets to represent a collection, similar to how arrays are written. Now, if the client wants to query multiple resources at once, it sends a request to the URI where the last path element is the name of the resources in the example, `users`. However, this URI is not only used to query but also to create new resources. To create a new resource, the client sends an HTTP request with the POST verb to the server and submits a representation of the new resource to be created in the content area of the request message. It sends this request to the same URI `users`. Now, let's look at the access to the collection resources in the source code.

9 Query Parameters

Query Parameters

- Query parameters are used for filtering a collection

- `http://example.org/users?firstname=James`

Closely related to the so-called collections is the question of how we can filter out the resources of interest to us from the potentially large number of resources using filter criteria. For this, there is the possibility of adding query parameters after the path in the URIs. You can recognize this by the question mark, and after that, there are pairs of parameter names and values separated by an equal sign. If multiple filter criteria exist, they are separated by an ampersand. We return to filtering in another unit, which is completely dedicated to queries and filters.

10 Representations

Representations

- A resource is a collection of data attributes
 - Representations are used for the network transfer
 - XML, JSON, and CSV are frequently used
 - Images, Excel sheets, and PDF documents
-
- The representation format is called **media-type**
 - Clients request media types by header `Accept`
 - Servers respond with by header `Content-Type`

Next, let's look at how resources are represented as documents. The resource data must be able to be sent in some way as part of the HTTP messages. The technically correct term for this is representations. In practice, frequently used representations are used, for example, in XML, JSON, or even the very simple CSV format, which consists only of a comma-separated list of values. However, these are only some of the representation formats in which resources can be represented. Depending on the use case, defining resources as an image, for example, a diagram created under a certain aspect, makes sense. Excel, Word, or even PDF files would also be suitable here. The different representation formats all have unique names, and in the sense of HTTP standardization, these names are called media types. The parameter under which the media type is sent as part of an HTTP message varies somewhat depending on the HTTP verb and whether it is a request message or a response message. For example, when the client sends a GET request to the server, it submits a list of desired media types under the `Accept` header as part of the HTTP request. For example, the client can specify that it wants the list of resources of type `Person` in either JSON or XML format. The client must then also cope with both possible responses. The server then responds to such a request with the resources and in one of the desired representations, which of the two representations it chooses is up to it in the end. It must inform the client which formats it has selected as part of the HTTP response message with the `Content-Type` parameter. In the case of a POST request sent by the client to the server, the client then transmits data to the server. Accordingly, in this case, the client must tell the server which format the data is represented via the `Content-Type` parameter in the HTTP request message.

11 XML Representations

XML Representations

```
1 <person>
2   <firstName>James</firstName>
3   <lastName>Bond</lastName>
4   <birthDate>1960-06-23T00:00:00+02:00</birthDate>
5 </person>
```

- JAXB API to serialize and deserialize Java objects
- Serialization: Create a representation
- Deserialization: Create a Java object

We have already seen in the last examples how to work with JSON. RESTEasy and the Jackson Library go along very well here. Now, let's deal with the XML data format. For example, the XML representation of a resource of the type Person could look like what you can see on the slide beside me. The start of the representation is initiated by the tag Person, followed by the three attributes we want to convey to the resource Person here. The value is framed in each case by tags corresponding to the attribute's name. The end of the representation is marked by the closing Tag Person. XML as a data format can also be generated relatively easily in Java. Up to Java 8, the JAXB library was part of the SDK for this purpose. In the higher versions, JAXB must be integrated into the project, for example, via a Maven dependency. This library then supports the transfer of a Java object into the XML format, i.e., the serialization and the reverse direction, i.e., the creation of a Java object from XML, i.e., the deserialization.

12 Using XML representations in JAX-RS

Using XML representations in JAX-RS

You have to annotate your Java class:

```
1 @XmlRootElement
2 public class Person
3 {
4     private String firstName;
5     private String lastName;
6     private LocalDateTime birthDate;
7
8     public Person() {
9         super();
10    }
11
12    // Getter and Setter methods
13 }
```

- `XmlRootElement` is the marker for JAXB
- New `MessageBodyWriters` necessary for collections

To generate such a representation in XML format, one must provide the corresponding Java class with the annotation `XmlRootElement`. Make sure that the Java class that represents a resource has an empty constructor in any case so an empty instance of this class can be created first when deserializing. Getter and setter methods must exist for the attributes to be part of the XML representation. Further Java annotations allow the translation process between Java objects and XML to be controlled even more finely. We will not go into this further here for the time being but will discuss this later in an exercise if required. Unfortunately, handling collection responses is not easy in XML, and I will demonstrate this live on the computer.

13 Screencast

Screencast

- Produces and consumes
- `MessageBodyWriter`
- Show how to use in Postman

Let's have a look at the implementation of this media type decision. You can see here the producers and the consumes annotations. And normally it would be sufficient just to have one media type definition here, but in this example you can see we have a collection of media types. So we have JSON and XML. And this is true for all the endpoints except the delete, which does not consume anything. You can also see here in the signature or in the list of parameters, everything is the same. So we have here the university DTO object and REST-easy in combination with Jackson takes care of serializing and deserializing the incoming or outgoing representations into the specific objects here. In Postman you can make use of this media type by setting this accept header. So when you send a GET request, you define this accept header here and with application JSON you get this result. And by using XML here, you get an XML representation. And for GET this is easy, but for GET collection, switching to XML produces a correct XML representation, but on the backend side it is necessary to implement something for this. And this is implemented here in this util package. So we need a specific message body writer in case of XML. And this message body writer classes, actually it is an interface here that is provided by Jakarta. And you have to implement two methods here. The first one is `writable`. This method is called to by REST-easy to select the correct message body writer class. So you have to implement some logic here based on the given parameters to decide whether or not this implementation is ready to produce the media type that you want to have. And the other method is `writ2` and this actually then creates this representation. And in this method here, this is very straightforward. We have a, we manually, let's say manually write this universities tag, so the collection tag, which you can see here, universities. We write this manually to the output stream and then we iterate over the list of all results and create an XML representation of every university. And at the end, we close this tag

again. So this is how this, let's say, message body writer thing can be implemented when you want to produce something that is not supported by default.

14 Summary

Summary

- You have learned about resources
- You have learned about good and bad URIs
- You have learned about data representations
- You know how to consume and produce resources

To wrap up today's session, let's briefly summarize what you've learned. First, we explored the concept of resources—the fundamental building blocks of Web APIs. You now understand how resources represent entities in your application and how they are identified through URIs. Next, we discussed what makes a URI good or bad. A well-designed URI should be intuitive, meaningful, and consistent, enhancing the usability and maintainability of your API. We also covered data representations, emphasizing the importance of supporting multiple formats like JSON and XML to make your API flexible and accessible to various clients. Finally, you've learned how to consume and produce resources using JAX-RS annotations. This allows your API to handle different requests and responses effectively, adhering to Web API principles. With these skills, you are now well-equipped to design and implement robust Web APIs.

15 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.